

Hi Kiana and Valerie,

This is just a quick overview of the research presentation project we have to complete for CIS665. Our research project topic is: IoC - Inversion of Control (aka Dependency Injection) using Spring Boot and Java.

I think we can break this into 3 components as follows:

1. [Spring Framework Overview](#)
2. [Inversion of Control Overview](#)
3. [Application Demonstration supporting Spring and IoC](#)

I will tackle #3 if both of you would like to work on the first 2 parts.

Spring 5.0 Framework

The first will be to provide an overview of the Spring Framework. Spring is a tool that is widely used to create web applications and web services which are written in Java. The framework is very extensive with many component parts that can selectively be used when building an application. Think of these components or modules as technology stacks that an application can be integrated into an application to provide the given service. For example, Spring has a Security module that handles adding security and authentication to the application without having to know about the implementation details.

Links to reference material:

Spring Home

<https://spring.io/>

Spring Boot – this one is key as it will be what our project is based on

<https://spring.io/guides/gs/spring-boot/>

Spring Projects (modules)

<https://spring.io/projects>

Spring Wiki

https://en.wikipedia.org/wiki/Spring_Framework

Inversion of Control - IOC (aka Dependency Injection)

IOC is a central part of any Spring Boot application and it is supremely useful. In a nutshell dependency injection works like this.

In the old school of programming is you wanted to have your application interface and interact with say an SQL Server Database, you would write specific methods to do so but you would directly bind them to your application – basically build and compile them into your project. The problem with this is that if for whatever reason you have to change to a different database (e.g. Oracle DB) you will have to take your

application, refactor it, and build in the Oracle support. It's time consuming, a maintenance nightmare, and costs money.

With IOC, instead of building support directly into your application, you structure it such that you will tell the application what database for example it will use at startup. Further, you build a common *interface* to support the database and then your application doesn't even know what database is in use. There would be nothing in the Java code at the application layer that would indicate which database is active.

This is very easy to support given the object-oriented nature of Java and the use of *interfaces*. The interface would be common to any database that could conceivably be used. That is, the implementation details supporting a DB must all support the common interface. A class would be created for each database which could be instantiated at start-up. The only other piece would be how to indicate what DB object to use. This is done using Spring *annotations*. Annotations are hints to the Java compiler which allows it to bring in additional support classes/profiles/definitions that aid in the dynamic configuration of the application at startup.

So, for the database example there would be annotation(s) in the Java code that at startup would for example instantiate a database object support Oracle. The rest of the application Java code knows nothing of Oracle. This code simply makes database calls to the instantiated object using the common interface and lets the object itself take care of the details.

If you later need to switch to a different database – the main application code doesn't change at all. Only the annotation to indicate the new database class would change. It is a very cool concept – highly flexible – and this is just a simple example.

Links to reference material:

Spring Wiki:

[https://en.wikipedia.org/wiki/Spring_Framework#Inversion_of_control_container_\(dependency_injection\)](https://en.wikipedia.org/wiki/Spring_Framework#Inversion_of_control_container_(dependency_injection))

Baeldung (this guy is an expert on Spring)

<https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>

Spring Annotations

<https://www.baeldung.com/spring-boot-annotations>

Spring IOC Demo Application

The third part of the presentation would be to demonstrate the Spring IOC concept by an actual demo application. The application could use several mock classes supporting various databases, extending from the discussion above, and could demonstrate how IOC is supported within the application by code and how to dynamically change the functionality of the application by changing a profile configuration annotation. The demo would be very useful in showing the power of this feature and why it was selected as a topic.